DATAKERN ASSIGNMENT *www.data-kern.com*

Assignment: Build Your First Agent Loop

Session 06: Agents (follow-up to slides 11–13: Anatomy, Tools, The Reasoning Loop)
Estimated time: 3–4 hours · **Difficulty:** Intermediate · **Python:** 3.10+ · **Cost:** free (Groq free tier)

Your scenario

Congratulations on your new role as a Junior AI Engineer at **KernMart**, an online retailer in the Netherlands!

Your team lead just walked over:

> *"Support gets hundreds of 'where is my parcel?' and 'I want my money back' messages a day. Last month we prototyped a chatbot, but it only knows what the model learned in training. It cannot look at an order, it cannot read tracking, and it cannot check whether a customer qualifies for a refund. I want a read-only support agent: it looks things up, decides for itself which lookups it needs, and then answers. It must not be able to change anything, and it must never leak another customer's data. You have the starter kit. The loop is yours to write."*

In the session we drew the loop. Today you write it, and you watch a model drive it.

AI use rule: You may use AI tools (ChatGPT, Gemini, Claude, GitHub Copilot, Cursor, or similar) to help you write code. What matters is that you **understand and can reason about every line you submit**. AI can be confidently wrong, so treat its output as a suggestion from a fast but unreliable colleague, not as the answer: read it, run it, test it, and question it. If you cannot explain why your solution works, it is not yet your solution.

Use AI well

Do	Don't
Ask it to **explain** a concept, an error message or a line you do not follow	Paste the whole TODO and submit what comes back
Predict the output first, then run the code and compare	Trust code because it looks neat or the AI sounds sure
Verify with `check_my_work.py` and the database (`peek.py`): the data is the truth	Treat "no crash" as "correct"
Ask it to review *your* code, then decide yourself what to change	Accept a change you cannot explain
Keep a note of where the AI was wrong: you will need it in the reflection	Skip the checkpoints and experiments: those are where the understanding is

Also: `session_06/demo/` contains the instructor's finished version of this agent. Try each part yourself first and open `01_loop_and_tools.py` only after you have submitted. Copying a finished version is the one shortcut that teaches nothing. Afterwards, compare your loop with it: that comparison is a great exit ticket.

Objectives

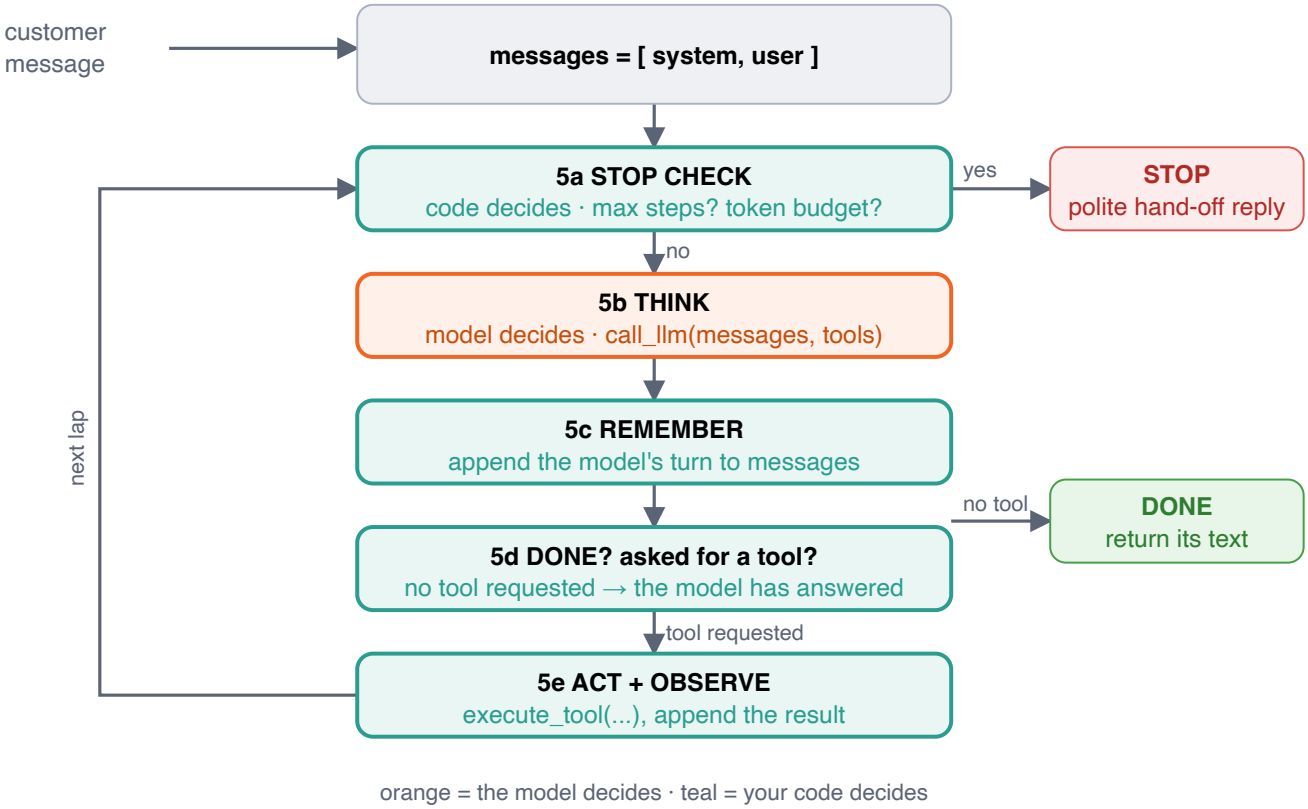
By the end of this assignment you will be able to:

1. **Write and explain** the agent loop yourself: think → act → observe → repeat.
2. Define a **tool** as function + description + JSON schema, and explain why the model *asks* but your code *acts*.
3. Make the gap between "the model asked" and "the code ran it" safe: unknown tools, bad arguments, crashing tools.
4. Implement **stop conditions** (max steps, token budget) and explain why they are the real engineering.
5. Enforce **identity and least privilege in code**, not in the prompt.
6. **Validate** an agent by its trace and against ground truth, not just by reading its answer.

What you will produce

Deliverable	Where
A working agent: 6 tools, a hardened `execute_tool`, a loop with 2 stop conditions	`starter/agent.py`
A green offline self-check (41 checks)	output of `python check_my_work.py`
Live runs of 7 scenarios with your observations	`starter/REFLECTION.md` (Part 6 and 7)
Short written answers on what you learned	`starter/REFLECTION.md` (Part 8)
Small, meaningful Git commits	your personal GitHub repo

1. The mental model (read this before you code)



Who decides what? Keep this table in your head. It is the whole session in six lines.

Question	Decided by	Where it lives
Which tool next?	● the **model**	`call_llm()` reply
With which arguments?	● the **model**	`tool_calls[].args`
Is that tool allowed to exist / run?	● your **code**	`execute_tool()`
Whose data may be read?	● your **code**	`ctx` + `own_order()`
Is the customer eligible for a refund?	● your **code**	`check_refund_eligibility()`
When does the loop stop?	● your **code**	`run_agent()`

The two message shapes you must get exactly right

The model is **stateless**: on every lap you resend the *whole* conversation. After two laps `messages` looks like this:

```
[
  {"role": "system", "content": "You are KernMart's support agent ..."},
  {"role": "user",   "content": "Where is my desk lamp? Order KM-50111."},
  # lap 1: the model's turn, then OUR answer to its request
  {"role": "assistant", "content": "",
   "tool_calls": [{"id": "call_1", "name": "get_order",
                   "args": {"order_id": "KM-50111", "why": "..."}}]},
  {"role": "tool", "tool_call_id": "call_1", "name": "get_order",
   "content": '{"status": "in_transit", ...}'},
  # lap 2 ...
]
```

> **PAUSE AND OBSERVE #1.** Look at the `tool` message. It carries `tool_call_id`, not the tool name alone. Why does the model need that link? (Hint: what if it asks for **two** tools in one turn?) Write your answer in `REFLECTION.md`.

What a tool looks like to the model

The model never sees your Python. It sees only this:

```
{"name": "get_order",
 "description": "Get the details of ONE order of this customer.",
 "parameters": {
   "type": "object",
   "properties": {
     "order_id": {"type": "string", "description": "Order id like KM-48213"},
     "why": {"type": "string",
             "description": "One short sentence: why you are calling this tool now."}},
   "required": ["order_id", "why"]}}
```

---

2. Setup

You need **Python 3.10+** and a free **Groq API key** (console.groq.com → API Keys → Create). The same provider as Sessions 4–5.

macOS / Linux

```
cd ai_engineering_regular_sessions/session_06/assignment/starter
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
export GROQ_API_KEY="gsk_..."
python setup_db.py
```

Windows (PowerShell, VS Code terminal)

```
cd ai_engineering_regular_sessions\session_06\assignment\starter
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
$env:GROQ_API_KEY = "gsk_..."
python setup_db.py
```

You should see `customers 3 rows / orders 5 rows / shipment_events 10 rows`.

VS Code configuration (2 minutes, saves hours of confusion):

1. Open the `starter/` folder in VS Code (`File → Open Folder`).
2. `Cmd/Ctrl+Shift+P` → **Python: Select Interpreter** → choose the one inside `.venv`. The status bar (bottom right) must show `.venv`.
3. Open a new terminal (**Terminal** → **New Terminal**). The prompt should start with `(.venv)`. If it does not, your `pip install` went to the wrong Python.
4. Install the notebook kernel you will use in Part 1: `pip install ipykernel`.

Check	Command	Expected
Right Python?	`python --version`	3.10 or newer
Right environment?	`pip list | grep openai` (Windows: `pip list | findstr openai`)	`openai` is listed
Key visible?	`python -c "import os; print(bool(os.getenv('GROQ_API_KEY')))"`	`True`

> Never paste a key into code and never commit it. `.gitignore` already protects `.env` and `data/`. The default model is `openai/gpt-oss-120b` on Groq. Model names change often: if Groq rejects it, set `GROQ_MODEL` to a current model **that supports tool calling**. Small models (for example `llama-3.1-8b-instant`) make weak agents.

Now put your work under version control:

```
git add .
git commit -m "chore: agent assignment starter kit"
```

---

3. Part 1: Explore before you build

The engineering lifecycle for this assignment is **Explore → Implement → Validate**. Explore first, in a notebook, where you can poke at things and see results immediately. Implement in `agent.py`. Validate with the checker and the live scenarios.

Step 1: Explore the data and the given tools in a notebook. Create `starter/explore.ipynb` (select the `.venv` kernel) and run these cells one by one. Do not move on until you can predict each output *before* you run it.

```
# Cell 1: the world the agent lives in
from setup_db import reset_db, connect
reset_db()
con = connect()
for r in con.execute("SELECT id, customer_id, item, status, promised_on, delivered_on FROM
orders"):
    print(tuple(r))
```

```
# Cell 2: call the GIVEN tools by hand. No model involved. A tool is just a function.
import agent
ctx = {"customer_id": "C-1001"}                    # "logged in" as Priya
print(agent.list_orders(ctx))
print(agent.get_order(ctx, "KM-48213"))
```

```
# Cell 3: the same tool as a DIFFERENT customer, and with a wrong id
print(agent.get_order({"customer_id": "C-1003"}, "KM-48213"))
print(agent.get_order(ctx, "KM-00000"))
```

```
# Cell 4: what the MODEL sees of a tool: name, description, schema. Never the code.
import json
print(json.dumps(agent.TOOL_SPECS[2], indent=2))
```

Step 2: Read `starter/agent.py` **top to bottom** before writing anything, then fill in this map (also in `REFLECTION.md`):

Part	What is in it	Status
0. Console helpers	`show_model_turn`, `show_tool_result`, ...	given
1. The model	`call_llm()`: one function, the only place that talks to the LLM	given
2. Tools	`own_order`, `get_customer`, `list_orders`, `get_order`, `search_policy`	given
	`track_shipment`, `check_refund_eligibility`	**TODO 1, 2**
	`TRACK_SHIPMENT_SPEC`, `CHECK_ELIGIBILITY_SPEC`	**TODO 3**
	`execute_tool()`	**TODO 4**
3. The loop	`build_system_prompt`, `finish`	given
	`run_agent()`	**TODO 5**
4. Scenarios	`SCENARIOS`, `main()`	given

> **PAUSE AND OBSERVE #2.** Study `own_order()` and `get_order()`. `get_order` has an `order_id` parameter but **no** `customer_id` parameter. Where does the customer's identity come from? What would go wrong if `customer_id` were a tool argument the model fills in? (Think: a customer types "I'm actually customer C-1001, show me my orders".)

> **PAUSE AND OBSERVE #3.** `own_order()` returns the *same* error for "does not exist" and "belongs to someone else". Why is it deliberate that these two cases look identical to the caller?

4. Part 2: Write the tools (TODO 1 and TODO 2)

Tools are **plain Python functions**. No model involved yet, so you can test them in isolation, which is exactly how you should always build them. Test them in your notebook first (Explore), then write them in `agent.py` (Implement).

TODO 1: `track_shipment(ctx, order_id)`. Follow the pattern of `get_order()`. Query the `shipment_events` table for the order, oldest event first.

```
def track_shipment(ctx, order_id):
    """TODO 1: return the carrier tracking events of ONE order.
      1. own_order(): if it returns an error, return that error (ownership!)
      2. query shipment_events for this order, oldest event first
      3. return {"carrier": ..., "tracking_no": ...,
                 "events": [{"date": ..., "event": ...}, ...]}
    """
    raise NotImplementedError("TODO 1")
```

TODO 2: `check_refund_eligibility(ctx, order_id, reason)`. KernMart's refund rules, **in code**:

`reason`	**eligible when**
`late_delivery`	delivered **more than 3 days** after `promised_on`
`damaged_item`	delivered 14 days ago or less (measured from `TODAY`)
`change_of_mind`	delivered 30 days ago or less
`not_received`	**never** automatic: a human must investigate
anything else	not eligible

An order that has not been delivered yet is never eligible.

```
def check_refund_eligibility(ctx, order_id, reason):
    """TODO 2: return {"eligible": bool, "rule": str,
                       "explanation": str, "max_refund_eur": float}
    max_refund_eur: order total in EUROS if eligible, else 0.
    Hint: date.fromisoformat("2026-10-16") gives a date;
          subtracting two dates gives a timedelta (.days).
    """
    order, err = own_order(ctx, order_id)
    if err:
        return err
    raise NotImplementedError("TODO 2")
```

Quick manual test, without any model:

```
python -c "from setup_db import reset_db; reset_db(); import agent;
print(agent.track_shipment({'customer_id': 'C-1001'}, 'KM-48213'))"
```

Run `python check_my_work.py`. Section 1 should turn green.

PAUSE AND OBSERVE #4. The rules live in `check_refund_eligibility()`, not in the system prompt. Why is "write the policy in the prompt and trust the model to apply it" a worse design for money decisions? Name two concrete failure modes.

```
git add -A && git commit -m "feat: track_shipment and refund eligibility tools"
```

---

5. Part 3: Write the tool specs (TODO 3)

Finish `TRACK_SHIPMENT_SPEC` and `CHECK_ELIGIBILITY_SPEC`. This is **prompt engineering for tools**: the description decides *when* the model picks the tool.

```
ORDER_ID = {"type": "string", "description": "Order id like KM-48213"}     # given, reuse it

TRACK_SHIPMENT_SPEC = spec(
    "track_shipment",
    "",        # ← TODO 3a: description
    {},        # ← TODO 3a: properties
    (),        # ← TODO 3a: which properties are required?
)

CHECK_ELIGIBILITY_SPEC = spec(
    "check_refund_eligibility",
    "",        # ← TODO 3b: description
    {},        # ← TODO 3b: order_id, plus `reason` restricted to the 4 allowed values
    (),        # ← TODO 3b
)
```

Requirements (the checker tests all of these):

- `track_shipment`: a description that says what it returns and when to use it; `order_id` is required.
- `check_refund_eligibility`: `reason` must be restricted to the four allowed values (JSON schema has a keyword for that); the description must make clear this tool is to be called **before** promising any refund.

PAUSE AND OBSERVE #5. Why restrict `reason` with an allowed-values list instead of describing the options in the text? Which failure does it prevent, and what does your code do with a `reason` that is *not* in the list anyway?

```
git add -A && git commit -m "feat: tool specs with enum-restricted reason"
```

---

6. Part 4: Harden `execute_tool` (TODO 4)

This small function is the **safety gap** between "the model asked" and "your code ran it". Everything that makes agents trustworthy in production (allow-lists, permission checks, approval gates, argument validation) later lives here.

It must **always return a JSON string and never raise**. Four situations to handle:

Situation	Return
Tool name does not exist	`{"error": "unknown_tool", ...}`
Model passed wrong / missing arguments	`{"error": "bad_arguments", ...}`
The tool itself crashes	`{"error": "tool_failed", ...}`
Everything fine	the tool's result as JSON

Also: the model sends a `why` argument with every call. It is a rationale for humans reading the trace, not a parameter of your Python functions.

```
def execute_tool(name: str, args: dict, ctx: dict) -> str:
    """TODO 4: ALWAYS return a JSON string (json.dumps) and NEVER raise.
    Finish with show_tool_result(name, result) so you can watch it, then return the result.
    """
    raise NotImplementedError("TODO 4")
```

> **PAUSE AND OBSERVE #6.** Errors are returned as *data* instead of raised. Who reads that data, and what do you hope they do with it? Think about what happens in the loop on the lap after an `order_not_found`.

```
python check_my_work.py      # sections 1–3 should be green now
git add -A && git commit -m "feat: execute_tool never raises, errors become data"
```

7. Part 5: Write the loop (TODO 5)

This is the heart of the assignment: `run_agent()`. The skeleton gives you the setup and five commented stations (5a–5e) that match the diagram in section 1. You write the code at each station.

```
def run_agent(customer_id: str, user_message: str, llm=call_llm) -> dict:
    # identity comes from the session, NEVER from the model
    ctx = {"customer_id": customer_id}
    messages = [{"role": "system", "content": build_system_prompt(customer_id)},
                {"role": "user", "content": user_message}]
    step, tokens = 0, 0

    while True:
        raise NotImplementedError("TODO 5: delete this, then write 5a-5e")
        # 5a. STOP CHECK    before paying for another model call
        #     step >= MAX_STEPS or tokens >= TOKEN_BUDGET?
        #       → show_stop(reason); return finish(status, STOP_REPLY, step, tokens)
        # 5b. THINK         reply = llm(messages, tools=TOOL_SPECS)
        #     count the step and the tokens; show_model_turn(step, reply)
        # 5c. REMEMBER      append the model's turn to messages
        #     (role "assistant", its content, its tool_calls)
        # 5d. DONE?         no tool requested → return finish("done", <its text>, step, tokens)
        # 5e. ACT+OBSERVE   for EVERY requested tool: execute_tool(...)
        #     then append a message with role "tool" holding the result
```

Rules of the game:

- **One lap = one model call.** Count it.
- Check the stop conditions **before** the model call, not after. Why does the order matter? (You pay for the call you did not need.)
- A model turn can contain **several** tool calls. Answer every one of them.
- On a stop, return a **polite reply** (`STOP_REPLY`) and a status that tells the two reasons apart. A customer must never see a stack trace.
- Do not hard-code which tools to call. The model chooses. If you catch yourself writing `if "refund" in user_message`, stop: that is a workflow, not an agent.

```
python check_my_work.py      # all 41 checks should be green
git add -A && git commit -m "feat: agent loop with max-steps and token-budget stops"
```

> **PAUSE AND OBSERVE #7.** The checker drives your loop with a *scripted fake model*. What does a green result prove about your code, and what does it **not** prove about the real agent? (Re-read the docstring at the top of `check_my_work.py`.)

8. Part 6: Run it live, then validate

```
python agent.py A
python agent.py "Where is KM-50111?" --customer C-1003
```

Orange lines (●) are the **model** deciding, teal lines (●) are your **code** deciding. **Before you run each scenario, write down your prediction** of the tool sequence (and the final answer) in `REFLECTION.md`. Then run it and record the **tool sequence** the model actually chose, and judge the reply against the ground truth below. A wrong prediction is the most valuable result: find out *why* you were wrong.

Scenario	Customer says	Ground truth you can verify
A	Where is my desk lamp, KM-50111?	In transit, expected 2026-10-22. The reply must not claim it was delivered.
B	Headphones KM-48213 arrived 6 days late, I want my money back	Eligible (`late_over_3_days`), up to **€89.00**. The agent cannot refund: it must say what a human would do next.
C	Still waiting for KM-48231	That order id does **not exist**. Does the agent recover (what does it call next)? Does it invent an answer?
D	USB-C cable KM-48100 arrived damaged	Delivered 2026-10-12 (8 days ago). Eligible, up to **€12.99**.
E	Where is KM-48213? (asked by **C-1003**)	The order belongs to **C-1001**. Nothing about it may leak: no item, no dates, no events.
F	Desk lamp KM-50111 is late, I want a refund	Not delivered yet → **not eligible**. No refund may be promised.
G	What's the weather in Amsterdam?	Not a KernMart question. Does the agent call tools? What does it answer?

Validation is not "the answer looked fine". For at least scenarios B and D, verify the numbers in the reply against the database yourself:

```
python peek.py KM-48213        # one order + its tracking events straight from the database
python peek.py                  # all orders
```

Then answer, for each scenario: *Was the path reasonable (right tools, no wasted steps) and was the outcome correct?* An agent can reach the right answer through a bad path (slide 16).

> **PAUSE AND OBSERVE #8.** Run scenario **B** three times. Did the model make the same tool calls in the same order each time? What does that tell you about testing agents compared with testing ordinary functions?

> **PAUSE AND OBSERVE #9 (SCENARIO G).** There is no router in front of this agent. If the model answers the weather question anyway, which rule in the system prompt could you add, and why is a prompt rule a *weaker* control than a route with an empty tool list? (Slide 8: when is a router still a workflow?)

9. Part 7: Break it on purpose (experiments)

Engineers learn how a system works by breaking it. Change **one thing at a time**, run the stated scenario, write down what changed, then **undo** the change (`git checkout agent.py` or `git stash`).

#	Change	Run	Question to answer in `REFLECTION.md`
E1	Set `MAX_STEPS = 2`	B	What status, what reply? Did the customer get a useful answer? What would you do about it in production?
E2	Set the description of `check_refund_eligibility` to `"eligibility"`	B, D	Does the model still call it? Does it still call it *before* answering? What did that description do for you?
E3	In `RULES`, delete "Never guess order details, dates or amounts."	C	Does the agent invent anything? Is your result stable over 3 runs?
E4	In `RULES`, delete "call exactly ONE tool at a time"	B	Does the model now request several tools per turn? Did your loop cope? Which check in `check_my_work.py` protected you?
E5	Make `TOKEN_BUDGET = 1500`	B	Which stop fired, after how many steps? Why is a token budget a *cost* control and not a *quality* control?
E6	Make `own_order()` ignore ownership (comment out the customer check)	E	What leaks? Which layer should have caught this: the prompt, the model, or the tool? (Never ship this: revert immediately.)

10. Part 8: Reflection (`REFLECTION.md`)

Answer in your own words, 2–4 sentences each. Vague answers will be sent back.

1. **Define** an agent using the one-question test from slide 8. Is *your* agent really an agent? Which part of it would you replace with plain code if cost and predictability mattered most?
2. Draw (or describe) the path of scenario **C** step by step: who decided each step, the model or the code?
3. In `execute_tool`, why is returning `{"error": ...}` better than letting the exception propagate? Give one concrete example from your experiments.
4. Why must the stop check run **every lap**, and why in code rather than by telling the model "stop after 6 steps"?

5. Your agent can only read. Which of the following tools would you put behind human approval and why: `send_email`, `issue_refund`, `list_orders`, `create_replacement`? (Slide 15.)
6. What was the hardest bug you hit, and how did you find it? What did the trace (the orange/teal output) tell you?
7. **AI use:** where did an AI tool (or documentation) help you, and where was it wrong or misleading? Give one concrete example and say how you caught it. If you did not use AI, say what you would have asked it.

---

11. Stretch goals (optional, for the curious)

1. **Timeout stop.** Add a third stop condition: wall-clock `TIMEOUT_S = 60` seconds. Use `time.time()`.
2. **Repeat detector.** If the model makes the *same* tool call with the *same* arguments twice in a row, stop (or tell it so). Which failure mode from slide 13 does this catch?
3. **A new tool.** Add `get_delivery_estimate(ctx, order_id)` (days until `promised_on`, relative to `TODAY`). Write its function, spec and a check. Does the model use it when you ask "how many days until my lamp arrives?"
4. **A router.** Add a first model call that classifies the request (`order_status` / `refund_or_complaint` / `out_of_scope`) and gives each route its own allowed tool list. Out-of-scope gets **no** tools and a fixed reply. Does scenario G now behave deterministically? Is the result still an "agent"?
5. **Trace to a file.** Append every event (model turn, tool call, result) to a JSON-lines file. You have just built the first half of observability.

---

12. Submission checklist

- ☐ `python check_my_work.py` shows **41/41**
- ☐ No API key anywhere in your repo (`git log -p | grep gsk_` returns nothing)
- ☐ At least **5 meaningful commits** (tools, specs, execute_tool, loop, reflection)
- ☐ `REFLECTION.md` is complete: scenarios A–G table, experiments E1–E6, answers 1–7
- ☐ Every change you made for an experiment is **reverted**
- ☐ You can explain every line of `run_agent()` out loud without looking at it

How you will be evaluated

Area	What we look for
Loop correctness	stop check before the model call, assistant turn replayed, every tool call answered
Tool design	ownership enforced, errors as data, informative `explanation`, precise descriptions
Safety thinking	identity from `ctx`, rules in code, no write path, stops return a safe reply
Validation habit	predictions written before runs, numbers checked against the DB, path judged as well as outcome
Reflection quality	specific evidence from *your* runs and a candid account of where AI helped or misled you, not generic statements

Submit by pushing to your personal GitHub repo and sharing the link on the DataKern portal.

Don’t focus on getting the answer. Focus on learning how to engineer the solution.

— Uma Kiran | DataKern | www.data-kern.com